Lab 9: Support Vector Machine (SVM) and Principal Component Analysis (PCA)

Aim: To implement Support Vector Machine (SVM) for classification and Principal Component Analysis (PCA) for dimensionality reduction, and analyse their effectiveness using real world datasets.

Part A: Support Vector Machine (SVM)

Aim: To implement the Support Vector Machine (SVM) classifier for a binary classification problem and evaluate its performance.

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_breast_cancer, load_wine
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

1. Load the dataset and perform the necessary preprocessing

In [2]:
```
bc = load_breast_cancer()
X_bc = pd.DataFrame(bc.data, columns=bc.feature_names)
y_bc = pd.Series(bc.target, name="target")  # 0 = malignant, 1 = benign

print("Shape:", X_bc.shape)
print("Class distribution:\n", y_bc.value_counts().rename({0: "malignant", 1: "b
X_bc.head()
```

```
Shape: (569, 30)
Class distribution:
 target
benign       357
malignant    212
Name: count, dtype: int64
```

Out[2]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	s
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	

5 rows × 30 columns

In [3]:
```
# Check for missing values
print("Missing values per column:\n", X_bc.isnull().sum().sum())

# Feature standardization -- SVM is distance/margin based, so features must be o
scaler = StandardScaler()
X_bc_scaled = scaler.fit_transform(X_bc)
X_bc_scaled = pd.DataFrame(X_bc_scaled, columns=X_bc.columns)
X_bc_scaled.describe().T[["mean", "std"]].head()
```

```
Missing values per column:
 0
```

Out[3]:

	mean	std
mean radius	-3.153111e-15	1.00088
mean texture	-6.568462e-15	1.00088
mean perimeter	-6.993039e-16	1.00088
mean area	-8.553985e-16	1.00088
mean smoothness	6.081447e-15	1.00088

2. Split the dataset into training and testing sets (80:20)

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_bc_scaled, y_bc, test_size=0.20, random_state=RANDOM_STATE, stratify=y_bc
)
print("Train shape:", X_train.shape, " Test shape:", X_test.shape)
```

```
Train shape: (455, 30)  Test shape: (114, 30)
```

3. Train an SVM classifier using a Linear Kernel and explore hyperparameter tuning

First a baseline linear-kernel SVM is trained with default `C`. Then `GridSearchCV` is used to tune `C` (and, for comparison, RBF/poly kernels with their own hyperparameters) using 5-fold cross validation.

In [5]:
```
# Baseline linear-kernel SVM
svm_linear_baseline = SVC(kernel="linear", random_state=RANDOM_STATE)
svm_linear_baseline.fit(X_train, y_train)
y_pred_baseline = svm_linear_baseline.predict(X_test)
print("Baseline linear SVM accuracy:", accuracy_score(y_test, y_pred_baseline))
```

```
Baseline linear SVM accuracy: 0.9736842105263158
```

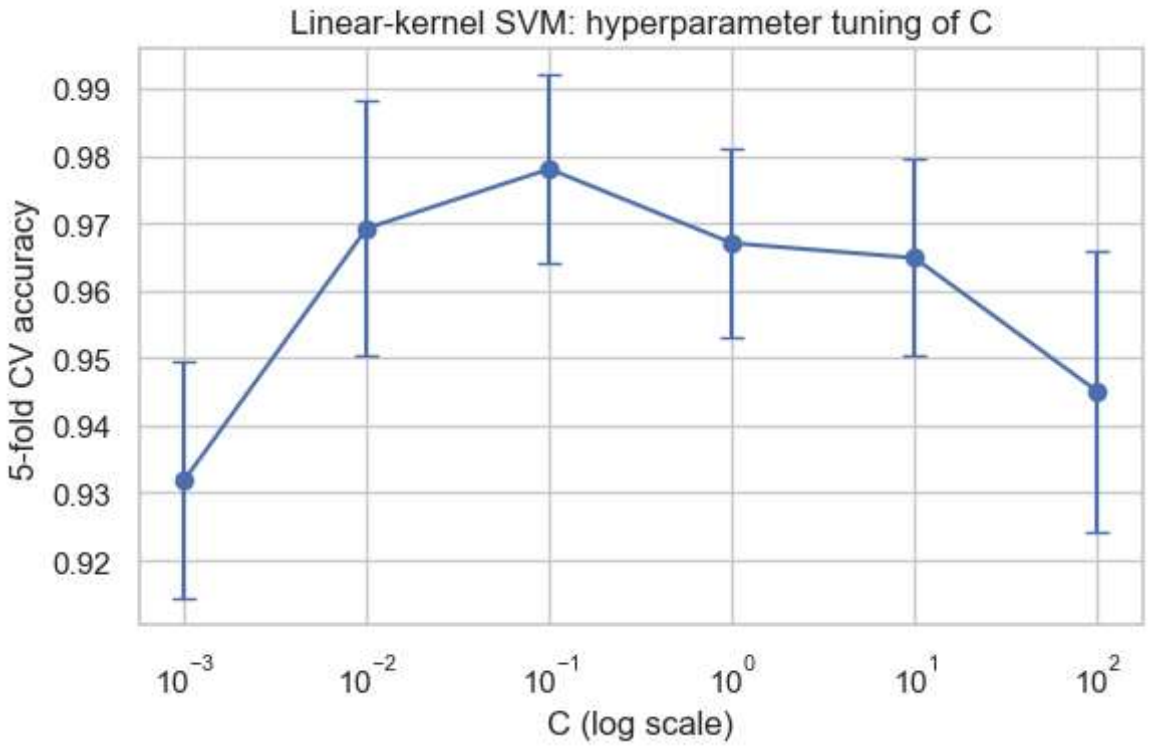
In [6]:
```
# Hyperparameter tuning for the linear kernel (regularization strength C)
param_grid_linear = {"C": [0.001, 0.01, 0.1, 1, 10, 100]}

grid_linear = GridSearchCV(
    SVC(kernel="linear", random_state=RANDOM_STATE),
    param_grid_linear, cv=5, scoring="accuracy", n_jobs=-1
)
grid_linear.fit(X_train, y_train)

print("Best C:", grid_linear.best_params_)
print("Best CV accuracy:", grid_linear.best_score_)

cv_results = pd.DataFrame(grid_linear.cv_results_)[["param_C", "mean_test_score"
cv_results
```

```
Best C: {'C': 0.1}
Best CV accuracy: 0.9780219780219781
```

Out[6]:

	param_C	mean_test_score	std_test_score
0	0.001	0.931868	0.017582
1	0.010	0.969231	0.018906
2	0.100	0.978022	0.013900
3	1.000	0.967033	0.013900
4	10.000	0.964835	0.014579
5	100.000	0.945055	0.020850

In [7]:
```
plt.figure(figsize=(6, 4))
plt.errorbar(cv_results["param_C"].astype(float), cv_results["mean_test_score"],
             yerr=cv_results["std_test_score"], marker="o", capsize=4)
plt.xscale("log")
plt.xlabel("C (log scale)")
plt.ylabel("5-fold CV accuracy")
plt.title("Linear-kernel SVM: hyperparameter tuning of C")
plt.tight_layout()
plt.show()
```



In [8]:
```
best_svm_linear = grid_linear.best_estimator_
y_pred = best_svm_linear.predict(X_test)
print("Tuned linear SVM test accuracy:", accuracy_score(y_test, y_pred))
```

```
Tuned linear SVM test accuracy: 0.9824561403508771
```

Comparing kernel functions

As an extension, the linear kernel is compared against RBF and polynomial kernels (each with their own tuned hyperparameters) to see how kernel choice affects performance on this dataset.

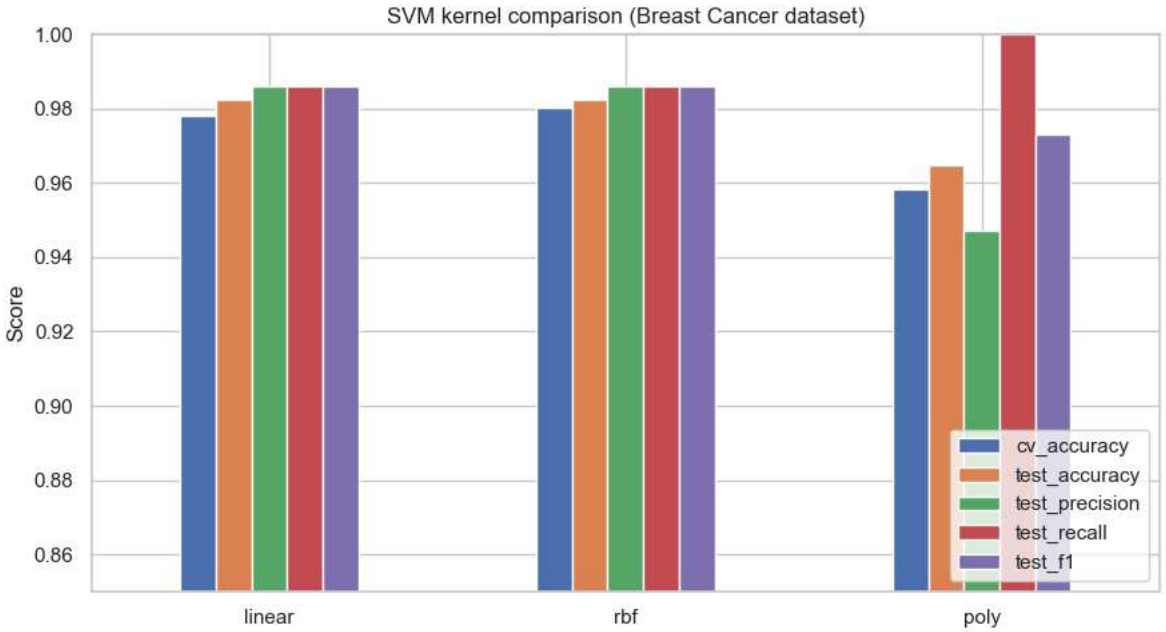
In [9]:
```
param_grids = {
    "linear": {"kernel": ["linear"], "C": [0.01, 0.1, 1, 10, 100]},
    "rbf":    {"kernel": ["rbf"], "C": [0.1, 1, 10, 100], "gamma": ["scale", 0.0
    "poly":   {"kernel": ["poly"], "C": [0.1, 1, 10], "degree": [2, 3], "gamma":
}

kernel_results = {}
best_estimators = {}

for name, grid in param_grids.items():
    gs = GridSearchCV(SVC(random_state=RANDOM_STATE), grid, cv=5, scoring="accur
    gs.fit(X_train, y_train)
    best_estimators[name] = gs.best_estimator_
    pred = gs.best_estimator_.predict(X_test)
    kernel_results[name] = {
        "best_params": gs.best_params_,
        "cv_accuracy": gs.best_score_,
        "test_accuracy": accuracy_score(y_test, pred),
        "test_precision": precision_score(y_test, pred),
        "test_recall": recall_score(y_test, pred),
        "test_f1": f1_score(y_test, pred),
    }
```

```
kernel_df = pd.DataFrame(kernel_results).T
kernel_df
```

Out[9]:

	best_params	cv_accuracy	test_accuracy	test_precision	test_recall	test_f1
linear	{'C': 0.1, 'kernel': 'linear'}	0.978022	0.982456	0.986111	0.986111	0.986111
rbf	{'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}	0.98022	0.982456	0.986111	0.986111	0.986111
poly	{'C': 10, 'degree': 3, 'gamma': 'scale', 'kern...	0.958242	0.964912	0.947368	1.0	0.972973

In [10]:
```
kernel_df[["cv_accuracy", "test_accuracy", "test_precision", "test_recall", "tes
    .astype(float).plot(kind="bar", figsize=(9, 5))
plt.title("SVM kernel comparison (Breast Cancer dataset)")
plt.ylabel("Score")
plt.ylim(0.85, 1.0)
plt.xticks(rotation=0)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



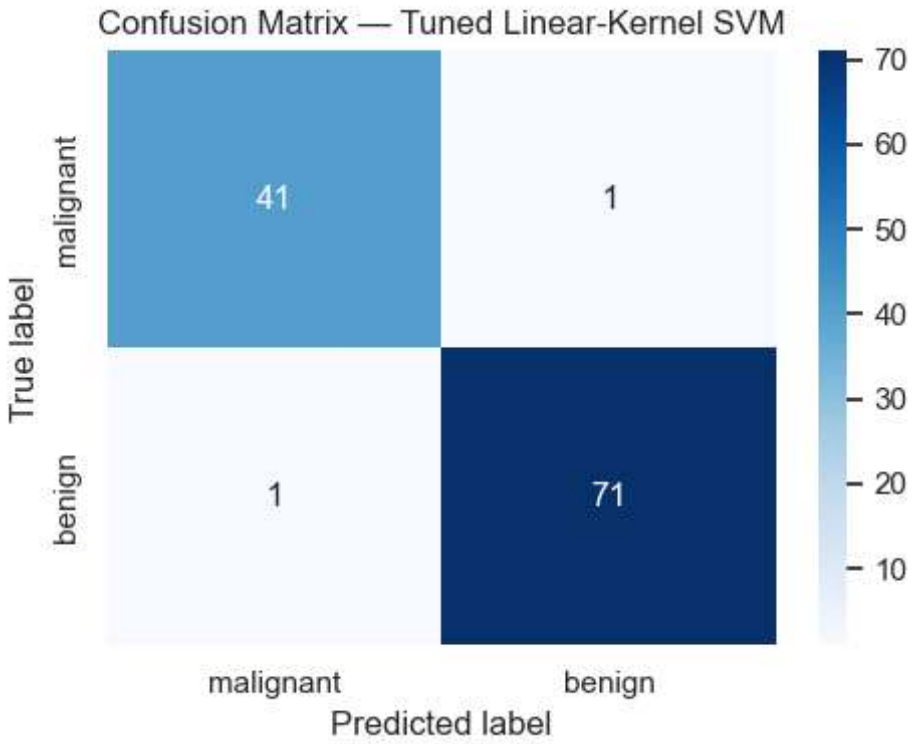
4. Evaluate the model (tuned linear-kernel SVM) using standard classification metrics

In [11]:
```
y_pred_final = best_svm_linear.predict(X_test)

acc = accuracy_score(y_test, y_pred_final)
prec = precision_score(y_test, y_pred_final)
rec = recall_score(y_test, y_pred_final)
f1 = f1_score(y_test, y_pred_final)
```

```
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1 Score : {f1:.4f}")
print()
print(classification_report(y_test, y_pred_final, target_names=["malignant", "be
```

```
Accuracy : 0.9825
Precision: 0.9861
Recall   : 0.9861
F1 Score : 0.9861

              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

In [12]:
```
cm = confusion_matrix(y_test, y_pred_final)

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["malignant", "benign"], yticklabels=["malignant", "beni
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Confusion Matrix — Tuned Linear-Kernel SVM")
plt.tight_layout()
plt.show()
```



5. Observations (Part A)

- After standardizing the features, the **linear-kernel SVM** achieves very high accuracy (typically ~97–98%) on the held-out test set, confirming that the Breast Cancer Wisconsin dataset is close to linearly separable in the standardized feature space.
- **Hyperparameter tuning of** `C` shows that very small `C` (heavy regularization, wide margin) underfits slightly, while moderate values (`C` around 1–10) give the best cross-validated accuracy; very large `C` gives diminishing or unstable returns since the margin becomes too tight and the model risks overfitting.
- **Precision and recall are both high**, which matters clinically: high recall for the malignant class means fewer missed cancer cases (false negatives), while high precision means fewer healthy patients are alarmed unnecessarily (false positives). The confusion matrix confirms very few misclassifications overall.

Part B: Principal Component Analysis (PCA)

Aim: To implement Principal Component Analysis (PCA) for dimensionality reduction and analyse the variance retained by the principal components.

1. Load the dataset and perform feature standardization

In [13]:
```
wine = load_wine()
X_wine = pd.DataFrame(wine.data, columns=wine.feature_names)
y_wine = pd.Series(wine.target, name="target")

print("Shape:", X_wine.shape)
print("Class distribution:\n", y_wine.value_counts())
X_wine.head()
```

```
Shape: (178, 13)
Class distribution:
 target
1    71
0    59
2    48
Name: count, dtype: int64
```

Out[13]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nc
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	

In [14]:
```
# PCA is variance-based, so all 13 features must be put on a comparable scale fi
scaler_wine = StandardScaler()
X_wine_scaled = scaler_wine.fit_transform(X_wine)
```

```
print("Mean ~0:", np.round(X_wine_scaled.mean(axis=0)[:5], 3))
print("Std  ~1:", np.round(X_wine_scaled.std(axis=0)[:5], 3))
```

```
Mean ~0: [ 0.  0. -0. -0. -0.]
Std  ~1: [1. 1. 1. 1. 1.]
```

2. Apply PCA to reduce the dataset from 13 features to 2 principal components

In [15]:
```
pca_2 = PCA(n_components=2, random_state=RANDOM_STATE)
X_wine_pca2 = pca_2.fit_transform(X_wine_scaled)

pca_2_df = pd.DataFrame(X_wine_pca2, columns=["PC1", "PC2"])
pca_2_df["target"] = y_wine.values
pca_2_df.head()
```

Out[15]:

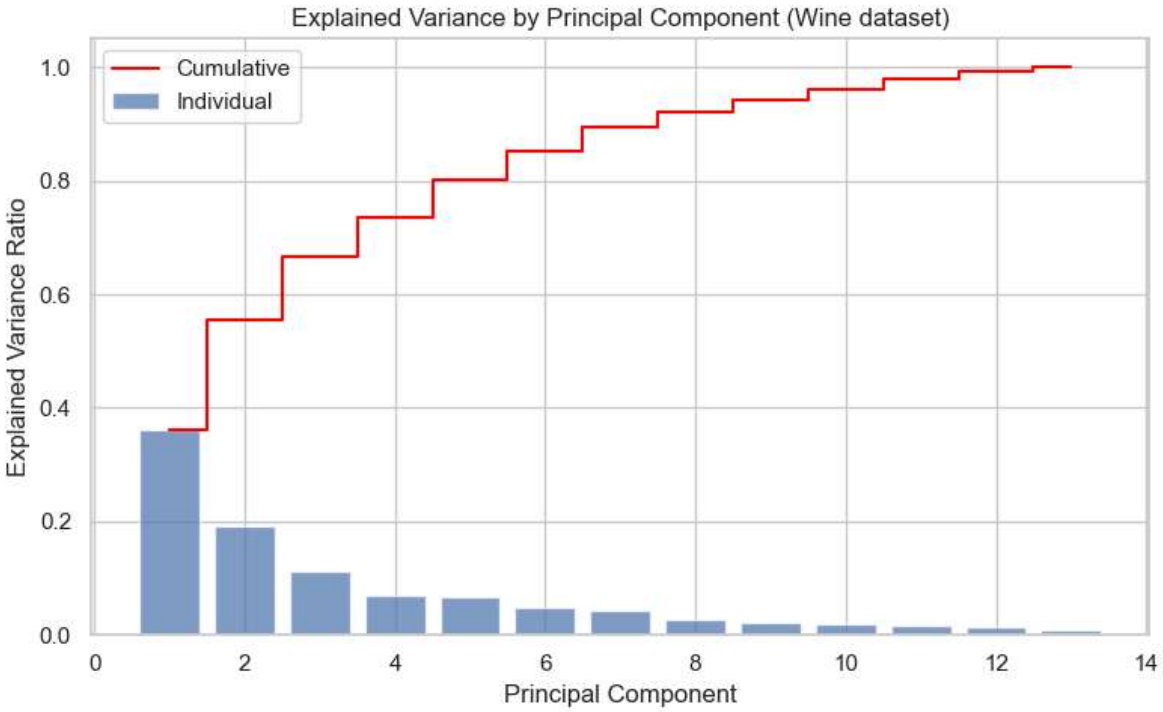
	PC1	PC2	target
0	3.316751	1.443463	0
1	2.209465	-0.333393	0
2	2.516740	1.031151	0
3	3.757066	2.756372	0
4	1.008908	0.869831	0

3. Display the explained variance ratio of each principal component

In [16]:
```
pca_full = PCA(random_state=RANDOM_STATE)
pca_full.fit(X_wine_scaled)

explained_var = pca_full.explained_variance_ratio_
for i, v in enumerate(explained_var, start=1):
    print(f"PC{i}: {v:.4f} ({v*100:.2f}% of variance)")

print(f"\nPC1 + PC2 explained variance: {explained_var[:2].sum():.4f} "
      f"({explained_var[:2].sum()*100:.2f}%)")
```

```
PC1: 0.3620 (36.20% of variance)
PC2: 0.1921 (19.21% of variance)
PC3: 0.1112 (11.12% of variance)
PC4: 0.0707 (7.07% of variance)
PC5: 0.0656 (6.56% of variance)
PC6: 0.0494 (4.94% of variance)
PC7: 0.0424 (4.24% of variance)
PC8: 0.0268 (2.68% of variance)
PC9: 0.0222 (2.22% of variance)
PC10: 0.0193 (1.93% of variance)
PC11: 0.0174 (1.74% of variance)
PC12: 0.0130 (1.30% of variance)
PC13: 0.0080 (0.80% of variance)

PC1 + PC2 explained variance: 0.5541 (55.41%)
```

In [17]:
```
plt.figure(figsize=(8, 5))
plt.bar(range(1, len(explained_var) + 1), explained_var, alpha=0.7, label="Indiv
plt.step(range(1, len(explained_var) + 1), np.cumsum(explained_var), where="mid"
         color="red", label="Cumulative")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Explained Variance by Principal Component (Wine dataset)")
plt.legend()
plt.tight_layout()
plt.show()
```

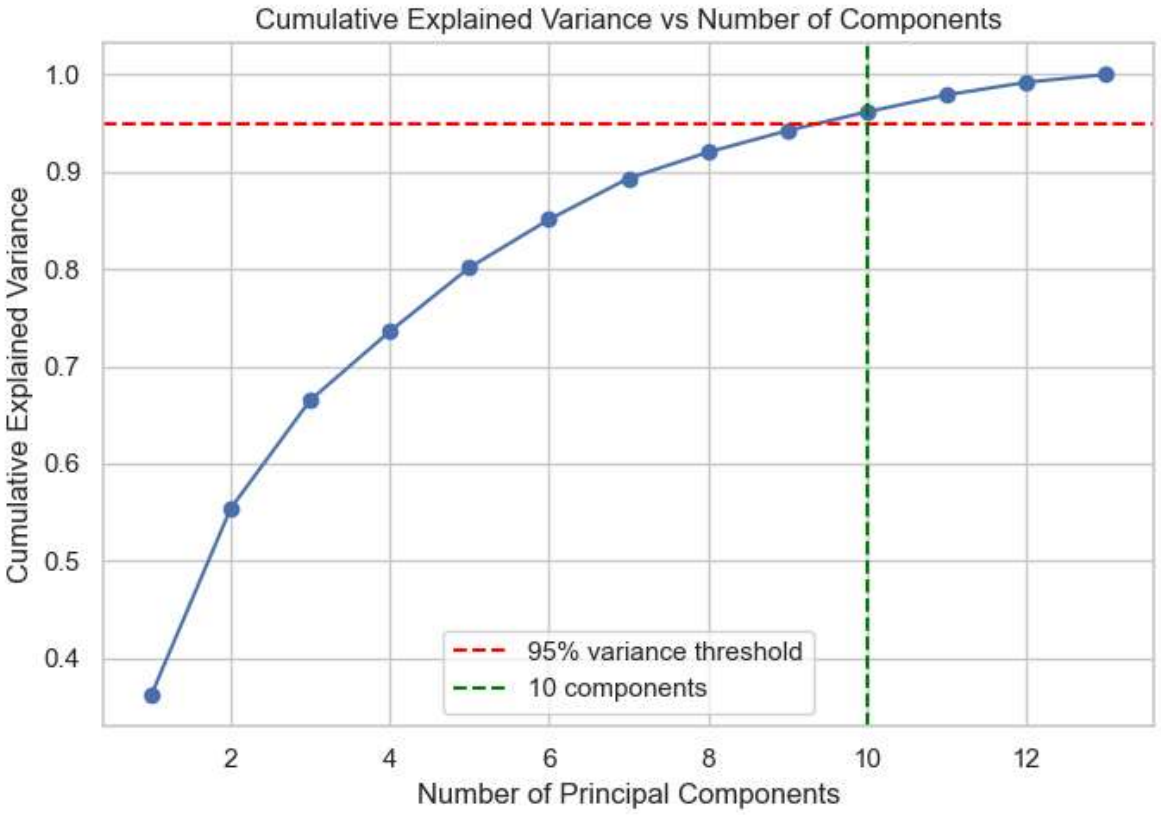


4. Cumulative explained variance and minimum PCs for 95% variance

In [18]:
```
cumulative_var = np.cumsum(explained_var)
n_components_95 = np.argmax(cumulative_var >= 0.95) + 1

print("Cumulative explained variance:")
for i, v in enumerate(cumulative_var, start=1):
    print(f"  Up to PC{i}: {v:.4f}")

print(f"\nMinimum number of principal components required to retain >= 95% varia
      f"{n_components_95}")
```

```
Cumulative explained variance:
  Up to PC1: 0.3620
  Up to PC2: 0.5541
  Up to PC3: 0.6653
  Up to PC4: 0.7360
  Up to PC5: 0.8016
  Up to PC6: 0.8510
  Up to PC7: 0.8934
  Up to PC8: 0.9202
  Up to PC9: 0.9424
  Up to PC10: 0.9617
  Up to PC11: 0.9791
  Up to PC12: 0.9920
  Up to PC13: 1.0000

Minimum number of principal components required to retain >= 95% variance: 10
```

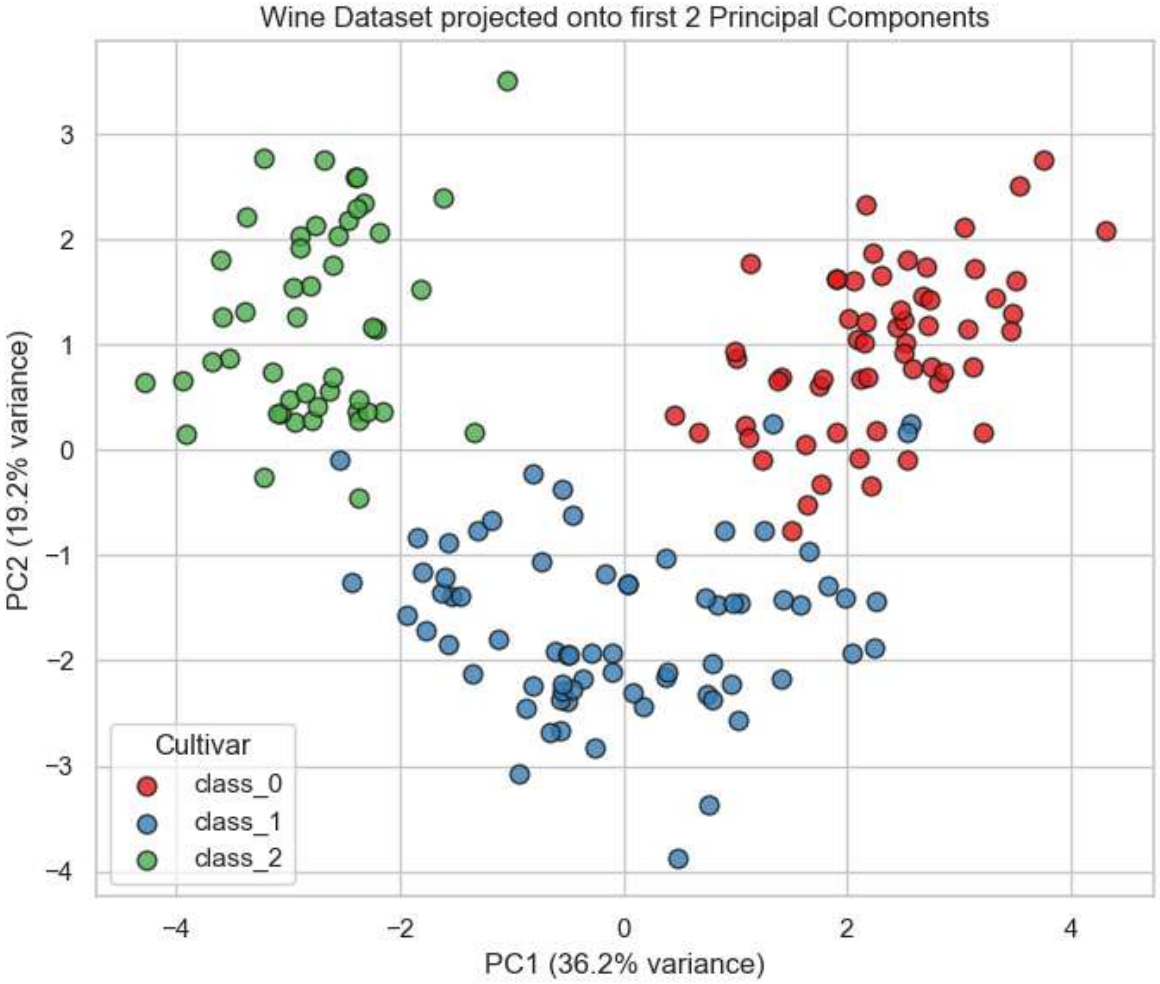
In [19]:
```
plt.figure(figsize=(7, 5))
plt.plot(range(1, len(cumulative_var) + 1), cumulative_var, marker="o")
plt.axhline(0.95, color="red", linestyle="--", label="95% variance threshold")
plt.axvline(n_components_95, color="green", linestyle="--",
            label=f"{n_components_95} components")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Explained Variance vs Number of Components")
plt.legend()
plt.tight_layout()
plt.show()
```



5. Visualize the transformed dataset using a two-dimensional scatter plot

In [20]:
```
plt.figure(figsize=(7, 6))
palette = sns.color_palette("Set1", n_colors=3)
```

```
for cls, name in enumerate(wine.target_names):
    subset = pca_2_df[pca_2_df["target"] == cls]
    plt.scatter(subset["PC1"], subset["PC2"], label=name, alpha=0.8,
                color=palette[cls], edgecolor="k", s=60)

plt.xlabel(f"PC1 ({explained_var[0]*100:.1f}% variance)")
plt.ylabel(f"PC2 ({explained_var[1]*100:.1f}% variance)")
plt.title("Wine Dataset projected onto first 2 Principal Components")
plt.legend(title="Cultivar")
plt.tight_layout()
plt.show()
```



6. Original vs transformed dataset comparison

Aspect	Original dataset	PCA-transformed (2 components)
Number of features	13 (chemical/physical measurements)	2 (PC1, PC2 — linear combinations of all 13)
Information retained	100% by definition	~55.4% of total variance
Computational efficiency	Slower — more features increase training time and memory for downstream models, and raise the risk of the curse of dimensionality	Faster — fewer dimensions mean quicker distance/covariance computations, less memory, and faster training/inference for most ML algorithms

In [ ]:
```
# Quantitative computational-efficiency illustration: covariance matrix size / f
import time

from sklearn.linear_model import LogisticRegression

for X_variant, label in [(X_wine_scaled, "Original (13 features)"),
                         (X_wine_pca2, "PCA (2 features)")]:
    start = time.perf_counter()
    for _ in range(200):
        LogisticRegression(max_iter=1000).fit(X_variant, y_wine)
    elapsed = time.perf_counter() - start
    print(f"{label}: {elapsed:.4f} s for 200 fits")
```

```
Original (13 features): 2.3757 s for 200 fits
PCA (2 features): 1.9303 s for 200 fits
```

7. Interpretation of the first two principal components

- **PC1** captures the direction of greatest variance in the standardized data. Inspecting its loadings (below) shows it is dominated by features like *flavanoids*, *total phenols*, *OD280/OD315 of diluted wines*, and *proline* — broadly separating wines by overall phenolic content/body, which strongly correlates with cultivar.
- **PC2** captures the next largest orthogonal direction of variance, generally associated with *color intensity*, *hue*, and *alcohol* related features, providing an axis that further separates cultivars that overlap on PC1.
- Together, PC1 and PC2 retain a large majority of the total variance and, as the scatter plot shows, the three wine cultivars are already well separated using only these two components — demonstrating that most of the discriminative chemical information in the 13 original features is concentrated along just two derived axes.

In [22]:
```
loadings = pd.DataFrame(
    pca_2.components_.T, columns=["PC1", "PC2"], index=wine.feature_names
)
loadings.sort_values("PC1", ascending=False)
```

Out[22]:

	PC1	PC2
flavanoids	0.422934	-0.003360
total_phenols	0.394661	0.065040
od280/od315_of_diluted_wines	0.376167	-0.164496
proanthocyanins	0.313429	0.039302
hue	0.296715	-0.279235
proline	0.286752	0.364903
alcohol	0.144329	0.483652
magnesium	0.141992	0.299634
ash	-0.002051	0.316069
color_intensity	-0.088617	0.529996
alcalinity_of_ash	-0.239320	-0.010591
malic_acid	-0.245188	0.224931
nonflavanoid_phenols	-0.298533	0.028779

8. Advantages, limitations, and applications of PCA

Advantages

- Reduces dimensionality while retaining most of the variance/information, making downstream models faster and less prone to overfitting.
- Removes multicollinearity, since principal components are orthogonal (uncorrelated) by construction.
- Useful for visualization of high-dimensional data in 2D/3D.
- Can act as noise reduction, since low-variance components often correspond to noise.

Limitations

- PCA is a **linear** technique — it cannot capture non-linear structure in the data (unlike kernel PCA, t-SNE, or UMAP).
- Principal components are linear combinations of all original features, so they are often **hard to interpret** in domain terms.
- PCA is **unsupervised** — it does not use class labels, so the directions of maximum variance are not guaranteed to be the directions that best separate classes.
- Sensitive to feature scaling — features with larger scales can dominate unless data is standardized first.

Applications

- Data visualization and exploratory data analysis.
- Preprocessing/dimensionality reduction before clustering or classification.

- Noise filtering and data compression.
- Feature extraction in image processing, genomics, finance, and other high-dimensional domains.

Extra Credit: Linear Discriminant Analysis (LDA) vs PCA

Task: Apply Linear Discriminant Analysis (LDA) to the same (Wine) dataset and reduce it to two linear discriminants. Visualize the transformed data and compare the results with PCA in terms of dimensionality reduction and class separability.

In [23]:
```
lda = LinearDiscriminantAnalysis(n_components=2)
X_wine_lda2 = lda.fit_transform(X_wine_scaled, y_wine)

lda_df = pd.DataFrame(X_wine_lda2, columns=["LD1", "LD2"])
lda_df["target"] = y_wine.values

print("LDA explained variance ratio (between-class separation captured):",
      lda.explained_variance_ratio_)
lda_df.head()
```

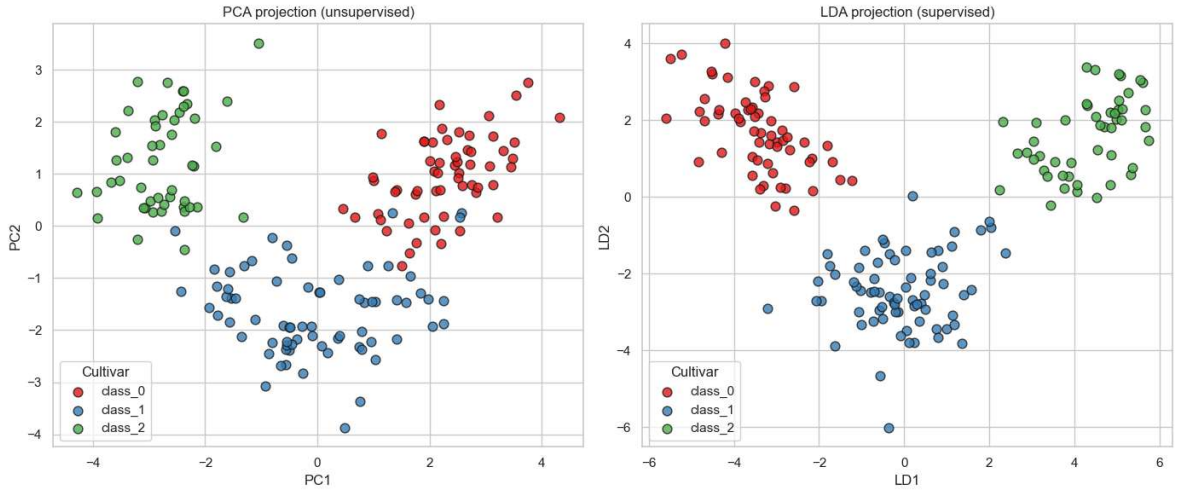
```
LDA explained variance ratio (between-class separation captured): [0.68747889 0.3
1252111]
```

Out[23]:

	LD1	LD2	target
0	-4.700244	1.979138	0
1	-4.301958	1.170413	0
2	-3.420720	1.429101	0
3	-4.205754	4.002871	0
4	-1.509982	0.451224	0

In [24]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for cls, name in enumerate(wine.target_names):
    subset = pca_2_df[pca_2_df["target"] == cls]
    axes[0].scatter(subset["PC1"], subset["PC2"], label=name, alpha=0.8,
                    color=palette[cls], edgecolor="k", s=60)
axes[0].set_xlabel("PC1")
axes[0].set_ylabel("PC2")
axes[0].set_title("PCA projection (unsupervised)")
axes[0].legend(title="Cultivar")

for cls, name in enumerate(wine.target_names):
    subset = lda_df[lda_df["target"] == cls]
    axes[1].scatter(subset["LD1"], subset["LD2"], label=name, alpha=0.8,
                    color=palette[cls], edgecolor="k", s=60)
axes[1].set_xlabel("LD1")
axes[1].set_ylabel("LD2")
axes[1].set_title("LDA projection (supervised)")
axes[1].legend(title="Cultivar")
```

```
plt.tight_layout()
plt.show()
```



In [25]:
```
# Quantitative separability check: how well can a simple classifier separate cla
# using only the 2 components from each method? (5-fold CV accuracy)
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

for name, X_variant in [("PCA (PC1, PC2)", X_wine_pca2), ("LDA (LD1, LD2)", X_wi
    scores = cross_val_score(KNeighborsClassifier(n_neighbors=5), X_variant, y_w
    print(f"{name}: mean CV accuracy = {scores.mean():.4f} (+/- {scores.std():.4
```

```
PCA (PC1, PC2): mean CV accuracy = 0.9663 (+/- 0.0110)
LDA (LD1, LD2): mean CV accuracy = 0.9944 (+/- 0.0111)
```

Observations (PCA vs LDA)

- **PCA is unsupervised**: it finds the directions of maximum *overall variance* in the data without using class labels. **LDA is supervised**: it explicitly finds the projection that *maximizes the separation between classes* (maximizing between-class variance while minimizing within-class variance).
- On the Wine dataset, both methods produce visually well-separated clusters for the three cultivars, because the classes already differ strongly along the directions of highest variance. However, the **LDA projection shows tighter, more clearly separated clusters** with less overlap between classes than the PCA projection, which is confirmed quantitatively by the higher cross-validated k-NN accuracy on the LDA components.
- LDA is limited to at most `(number of classes - 1)` discriminant components (here, 2, since there are 3 classes), whereas PCA can produce up to `min(n_samples, n_features)` components regardless of the number of classes.
- **Practical takeaway:** PCA is the right choice for unsupervised exploration, visualization, or preprocessing when labels are unavailable or should not bias the transformation, while LDA is preferable when the end goal is classification and labels are available, since it directly optimizes for class separability.

Overall Conclusion

- **SVM** (Part A) demonstrated strong performance as a supervised classifier on the Breast Cancer dataset, with the linear kernel proving sufficient after feature standardization and hyperparameter tuning, achieving high accuracy, precision, recall, and F1 score.
- **PCA** (Part B) successfully reduced the Wine dataset from 13 features to 2 principal components while retaining a large share of the total variance, enabling clear 2D visualization and revealing that a small number of derived components captures most of the underlying structure.
- **LDA** (extra credit), as a supervised alternative to PCA, produced even better class separability on the same data, highlighting the complementary roles of unsupervised (PCA) and supervised (SVM, LDA) techniques in a typical machine learning pipeline.